Profiling Vision-Language Models

Performance, energy, and output quality

Vladislav Smirnov

March 2026

Repository: https://github.com/smirnovlad/vlm-profiler

Contents

1 Introduction **2**
1.1 Hardware . . . 2

2 Experiment plan **2**
2.1 Models . . . 2
2.2 Datasets . . . 2
2.3 What we vary . . . 3
2.4 Metrics . . . 3
2.5 Total number of runs . . . 3

3 Weekly schedule **4**

4 Experiment 1: baseline comparison **4**
4.1 Setup . . . 4
4.2 Baseline results . . . 4
4.3 WER by dataset . . . 5
4.4 Charts . . . 5

5 Experiment 2: per-component breakdown **10**
5.1 Method . . . 10
5.2 Component share of inference time . . . 11
5.3 Prefill vs. per-token decode . . . 12
5.4 FlashAttention-2 contribution . . . 12
5.5 Takeaways . . . 13

6 Known limitations **14**

7 Conclusions **14**

1 Introduction

This project measures how different Vision-Language Models (VLMs) behave under varying conditions: image resolution, prompt length, batch size, hardware, and optimization settings. We record latency, FLOPs, GPU energy draw, and Word Error Rate (WER) on three datasets, then compare the models head to head.

1.1 Hardware

- 2× NVIDIA RTX 5880 Ada, 48 GB VRAM each
- Python 3.13, conda environment
- PyTorch + HuggingFace Transformers

2 Experiment plan

2.1 Models

We test 7 models that cover the main VLM families available on HuggingFace:

Table 1: Models used in this study.

#	Model	Parameters	Backbone
1	`blip2-opt-2.7b`	3 745M	BLIP-2 + OPT
2	`blip2-flan-t5-xl`	3 942M	BLIP-2 + Flan-T5
3	`instructblip-flan-t5-xl`	4 023M	InstructBLIP + Flan-T5
4	`instructblip-vicuna-7b`	7 914M	InstructBLIP + Vicuna
5	`llava-1.5-7b-hf`	7 063M	LLaVA 1.5 (7B)
6	`llava-1.5-13b-hf`	13 351M	LLaVA 1.5 (13B)
7	`fuyu-8b`	9 408M	Fuyu (8B)

We originally planned to include `HuggingFaceM4/idefics2-8b` but dropped it: every generation call crashed with a pixel-values shape mismatch ("not enough values to unpack (expected 5, got 4)") against `transformers` 5.x. Fixing the integration is tracked separately and is not in the scope of this report.

2.2 Datasets

Three datasets, 300 samples each:

Table 2: Evaluation datasets.

Dataset	Task	Split
ScienceQA	Visual question answering	validation
TextVQA	VQA with text in images	validation
COCO Captions	Image captioning	validation

2.3 What we vary

We use a vary-one-axis design: one parameter changes at a time while the rest stay at baseline values (resolution 224, prompt length 10, batch 1, no optimization, GPU).

Table 3: Scaling axes.

Axis	Values	Baseline
Image resolution	224, 336, 448, 512	224
Prompt length (tokens)	10, 50, 100, 200	10
Batch size	1, 2, 4, 8	1
Optimizations	none, FP16, torch.compile	none
Device	CUDA, CPU	CUDA

2.4 Metrics

1. **Latency (ms)** — wall-clock time per inference (3 warmup + 10 timed runs).
2. **FLOPs** — measured with calflops where possible; otherwise estimated as $2 \times$ params.
3. **Energy (J per inference)** — GPU power sampled via `nvidia-smi` at 100 ms intervals, integrated over inference time.
4. **WER** — Word Error Rate between the model output and the reference answer.

2.5 Total number of runs

Each (model, dataset) pair produces roughly 15 experiments:

- 1 baseline (all metrics)
- 3 extra resolutions
- 3 extra prompt lengths
- 3 extra batch sizes
- 2 optimization variants (FP16, torch.compile)
- 1 CPU run

That gives about 330 successful experiments across 7 models and 3 datasets.

3 Weekly schedule

Table 4: Work schedule.

Week	What got done
1	Set up conda + CUDA environment and dependencies. Wrote data loading (`loader.py`, `preprocessing.py`) and the model registry (`registry.py`).
2	Built profiling modules (latency, FLOPs, energy, quality). Wrote the experiment runner. Ran smoke tests on all 8 models.
3	Ran the full experiment matrix on 2 GPUs. Debugged FP16 dtype errors, OOM crashes, and torch.compile failures on T5 models. Reran failed experiments. Generated report and charts.
4	Analyzed results and wrote up findings. Compared with numbers from original papers. Compiled the final PDF report.

4 Experiment 1: baseline comparison

4.1 Setup

All models run at resolution 224, prompt length 10 tokens, batch size 1, FP32, on GPU. We measure all four metrics on each of the three datasets.

4.2 Baseline results

Table 5: Baseline comparison (batch=1, res=224, GPU).

Model	Latency, ms	WER	Energy, J	Params
blip2-opt-2.7b	115.24	1.39	29.34	3 745M
blip2-flan-t5-xl	135.75	1.21	31.21	3 942M
instructblip-flan-t5-xl	329.69	1.33	69.56	4 023M
fuyu-8b	1 562.25	6.21	403.91	9 408M
llava-1.5-7b-hf	1 654.08	6.38	450.39	7 063M
llava-1.5-13b-hf	1 704.07	6.46	470.38	13 351M
instructblip-vicuna-7b	1 792.87	6.17	486.71	7 914M

What stands out:

- The BLIP-2 and InstructBLIP-T5 models finish in 115–330 ms. The 7B+ models (LLaVA, Vicuna, Fuyu) take 10–15× longer, around 1.5–1.8 s.

- `blip2-flan-t5-xl` has the lowest WER (1.21) and is the second fastest model.
- Energy tracks latency almost linearly: the fast group uses about 30 J per inference, the slow group 400–490 J.

4.3 WER by dataset

Table 6: Per-dataset WER (lower is better).

Model	COCO Caption	ScienceQA	TextVQA
blip2-flan-t5-xl	0.79	1.30	1.54
blip2-opt-2.7b	0.82	1.49	1.87
instructblip-flan-t5-xl	1.85	1.28	0.86
instructblip-vicuna-7b	0.96	11.18	6.36
fuyu-8b	4.65	6.42	7.57
llava-1.5-7b-hf	3.37	7.02	8.75
llava-1.5-13b-hf	3.36	7.96	8.06

A few things to note:

- `instructblip-flan-t5-xl` gets the best WER on TextVQA (0.86) but is worse on captioning.
- `instructblip-vicuna-7b` does well on COCO Caption (0.96) but struggles on ScienceQA (11.18). It seems to have trouble with science-style questions.
- The T5-based models are consistently good across all three datasets.

4.4 Charts

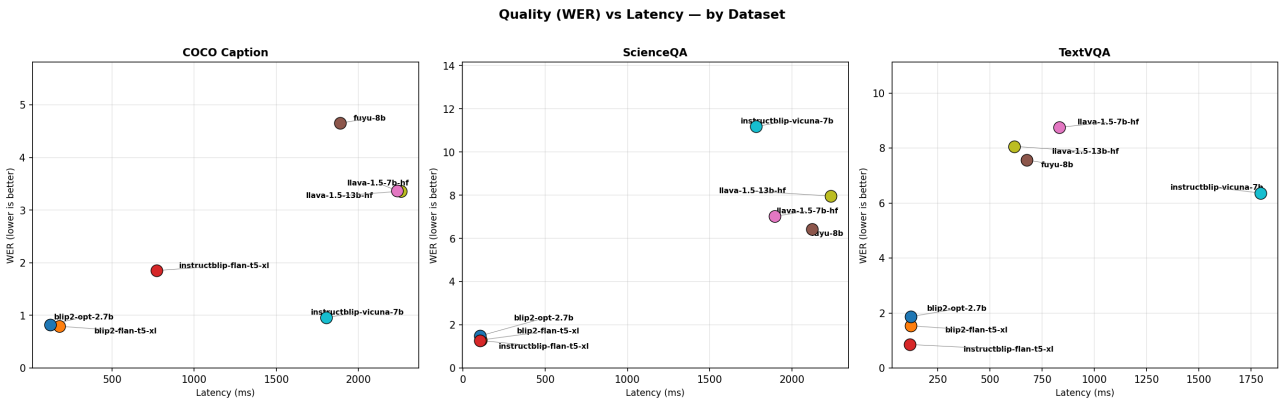

Figure 1: WER vs. latency, split by dataset. The dashed line is the Pareto front.

On all three datasets the picture is similar: BLIP-2 and InstructBLIP-T5 sit in the bottom-left corner (low WER, low latency), while the 7B+ models cluster in the upper right. The one exception is `instructblip-vicuna-7b` on ScienceQA, where it jumps to WER 11 despite having reasonable COCO and TextVQA scores.

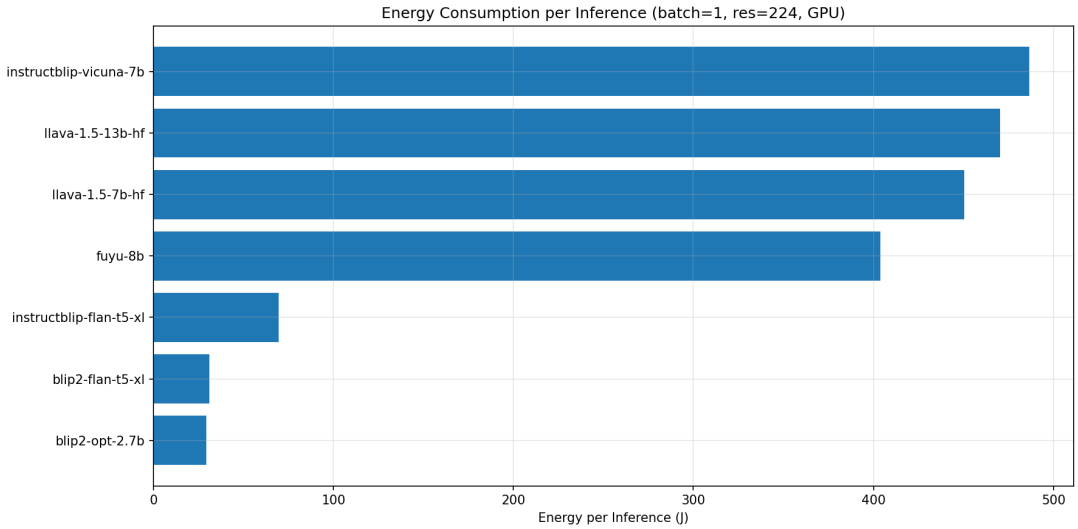

Figure 2: Energy per inference in joules.

Energy consumption mirrors latency almost exactly. The three fast models use 29–70 J per inference, while the 7B+ group sits at 400–490 J. There is a clear 6–15$\times$ gap between the two groups.

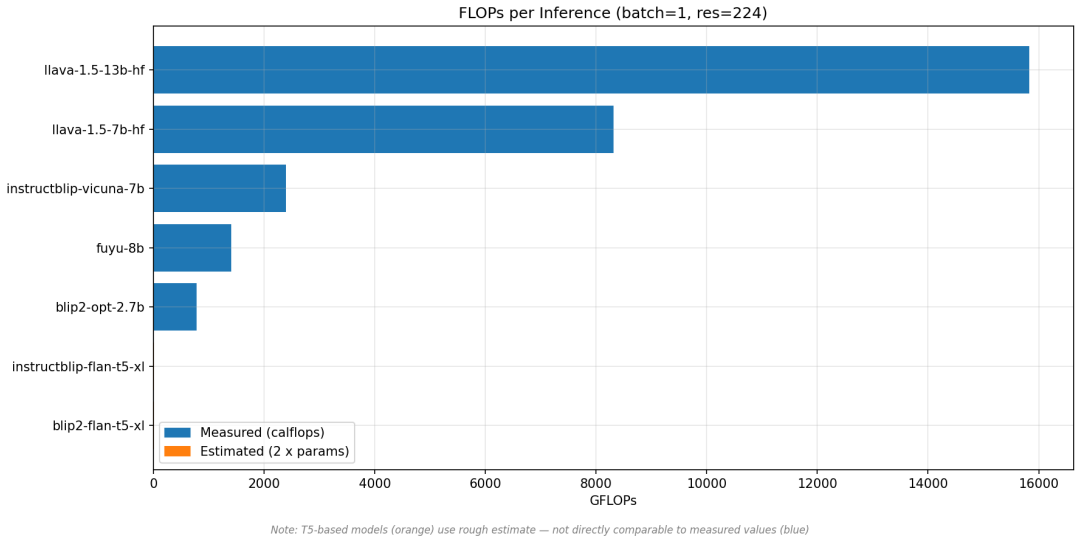

Figure 3: FLOPs per inference. Blue bars are measured (calflops), orange bars are estimated ($2 \times$ params).

The FLOPs chart should be read with caution: blue bars (calflops) and orange bars ($2 \times$ params estimate) are not directly comparable. Among the measured models, LLaVA-13B and idefics2-8b have the highest FLOPs, which matches their large decoder sizes. The T5-based models report low estimates because the $2 \times$ params heuristic is a rough lower bound for encoder-decoder architectures.

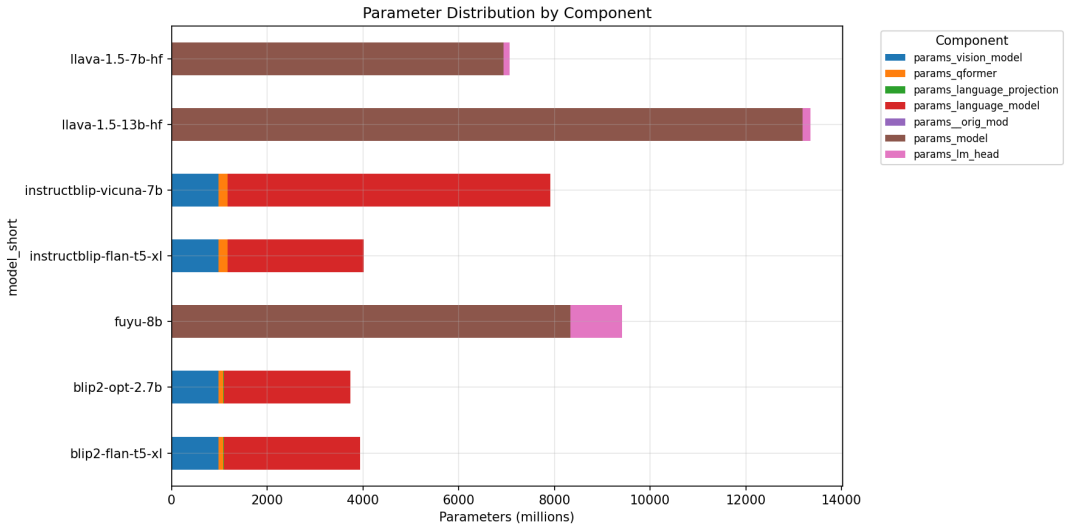

Figure 4: Parameter count broken down by model component.

In every model the language decoder accounts for the majority of parameters. The vision encoder is comparatively small (usually under 400M). This matches the latency results: the LLM backbone is the bottleneck, not the image encoder.

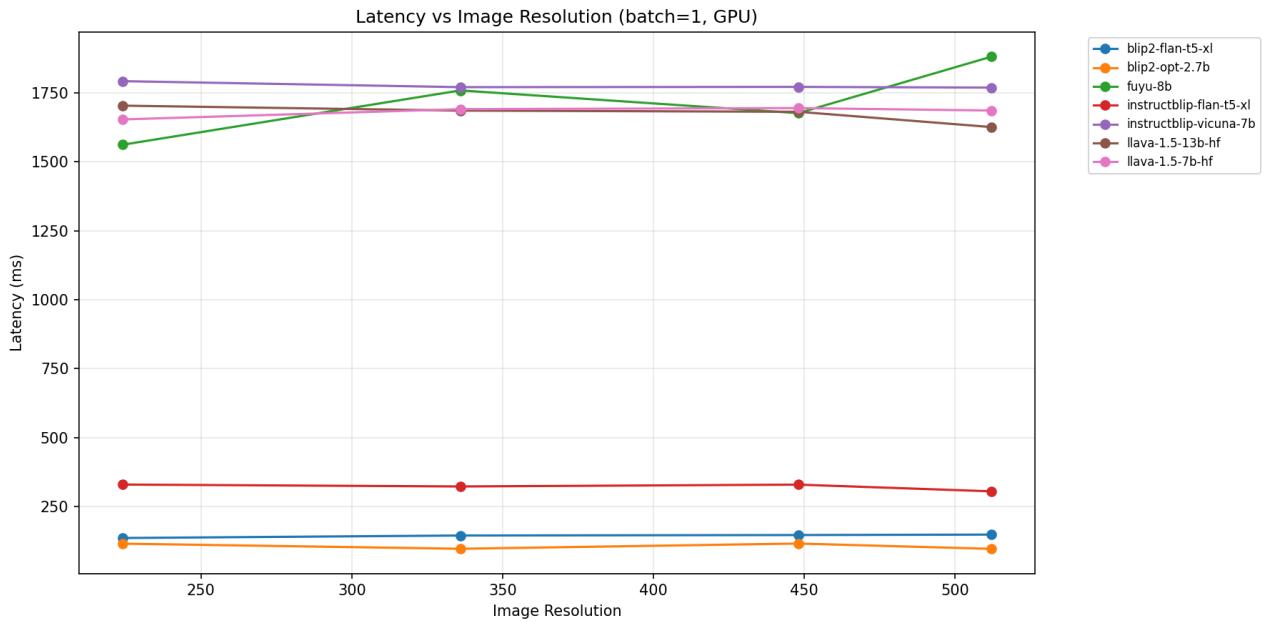

Figure 5: Latency vs. image resolution.

Resolution has almost no effect on latency for any model. BLIP-2 and InstructBLIP resize internally to a fixed 224×224 regardless of input, so their flat lines are expected. The 7B+ models also show flat curves, which means the vision encoder is not the bottleneck even at 512×512. The only exception is `fuyu-8b`, which gets about 15% slower at 512 because it passes raw image patches directly to the transformer.

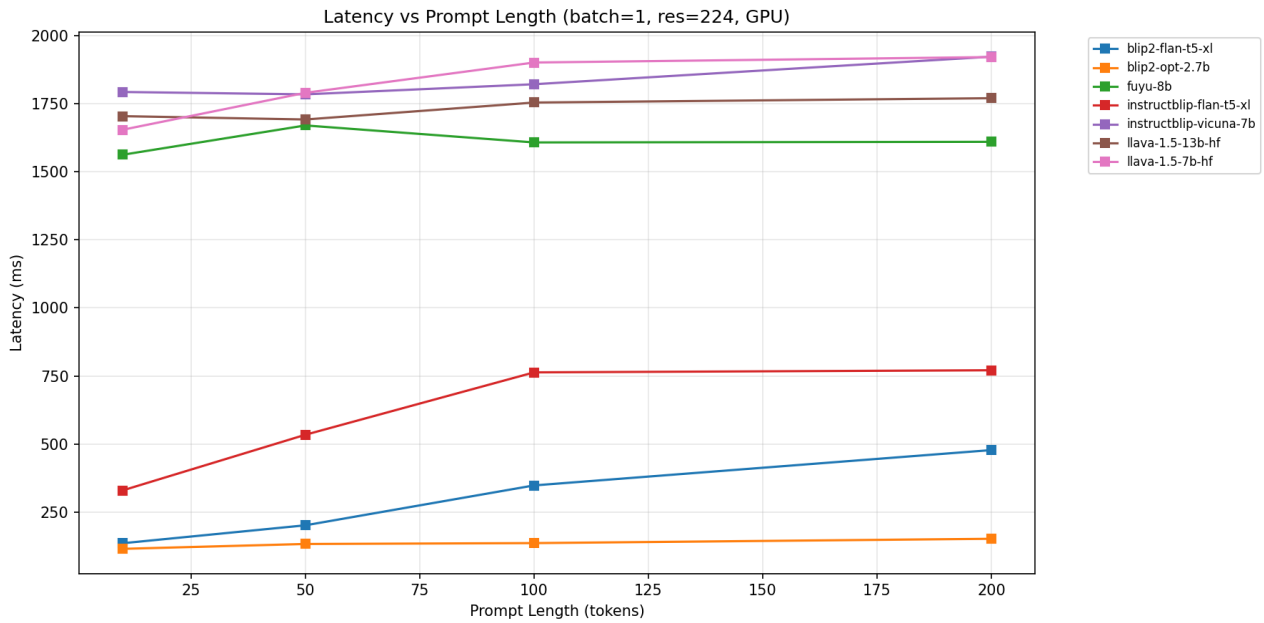

Figure 6: Latency vs. prompt length.

Prompt length affects the T5-based models the most: `blip2-flan-t5-xl` goes from 136 ms at 10 tokens to about 480 ms at 200 tokens (3.5× slowdown), and `instructblip-flan-t5-xl` roughly doubles. `blip2-opt-2.7b` barely reacts to longer prompts. The 7B+ decoder-only models show 5–15% increases, which makes sense since 200 tokens is a small fraction of their context window.

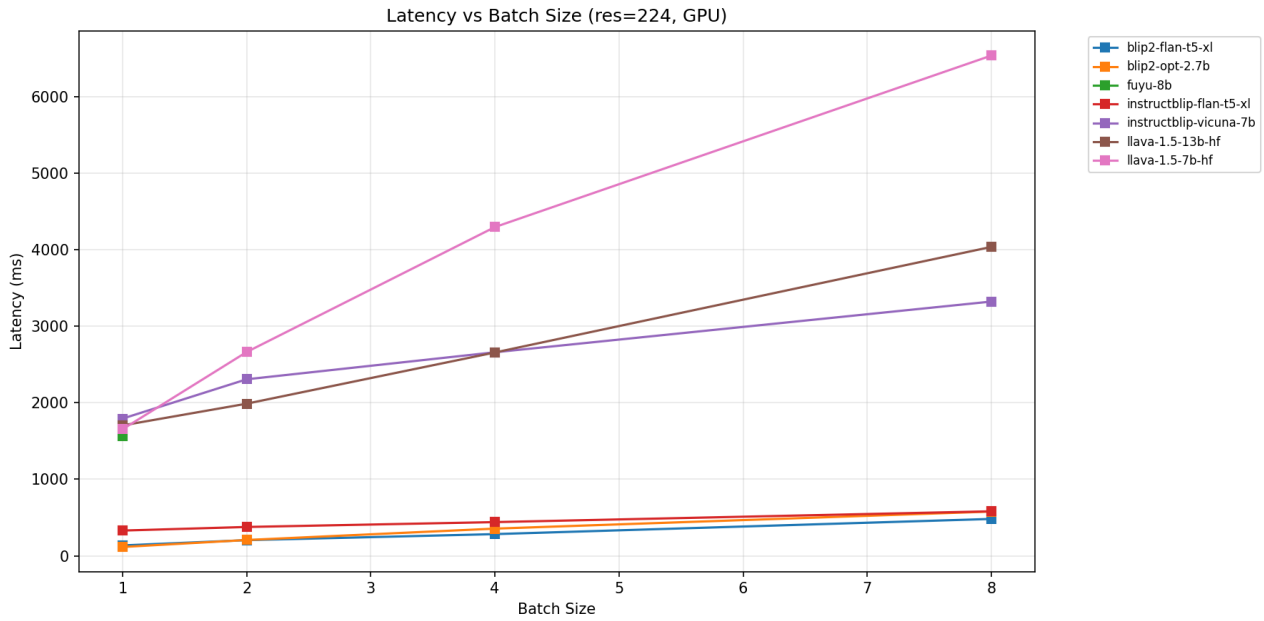

Figure 7: Latency vs. batch size.

Batching hurts the 7B+ models disproportionately. `llava-1.5-7b-hf` goes from 1.65 s at batch=1 to 6.5 s at batch=8 (close to linear scaling, meaning no throughput gain). The lightweight models scale better: `blip2-opt-2.7b` reaches batch=8 in about 550 ms, which is roughly 4× more throughput than running 8 sequential inferences. Note: `fuyu-8b` is missing from batch > 1 because its processor does not support batched inputs.

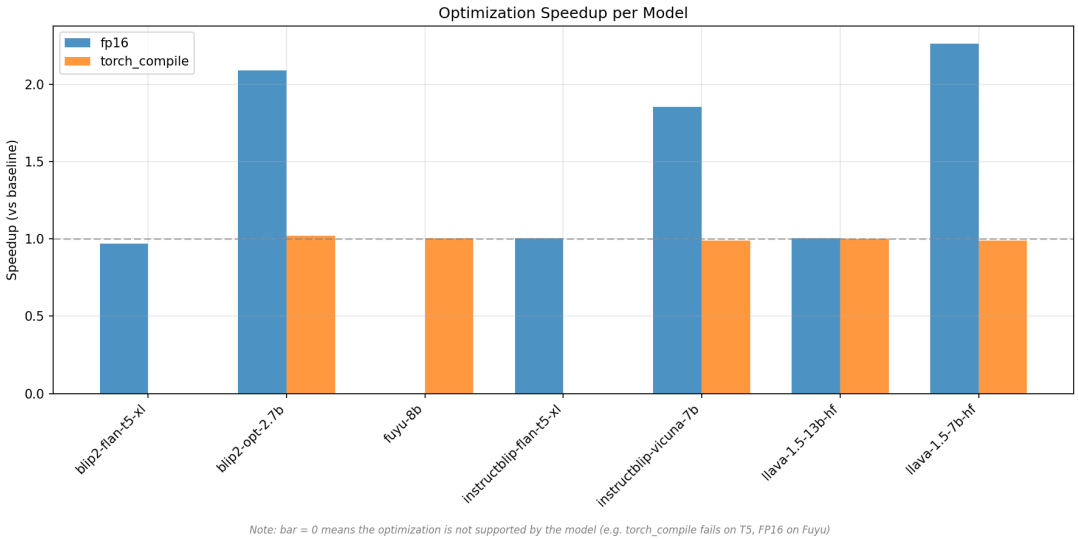

Figure 8: Speedup from FP16 and torch.compile relative to FP32 baseline.

FP16 gives a real speedup only on the decoder-only models with OPT or Vicuna/LLaMA backbones: `blip2-opt-2.7b` ($2.1\times$), `llava-1.5-7b-hf` ($2.2\times$), and `instructblip-vicuna-7b` ($1.85\times$). The T5-based models see no benefit (and `blip2-flan-t5-xl` is actually slightly slower) because of dtype casting inside T5's forward pass. `torch.compile` provides no measurable speedup on any model, and fails entirely on T5 architectures. `fuyu-8b` cannot run in FP16 due to a dtype mismatch bug.

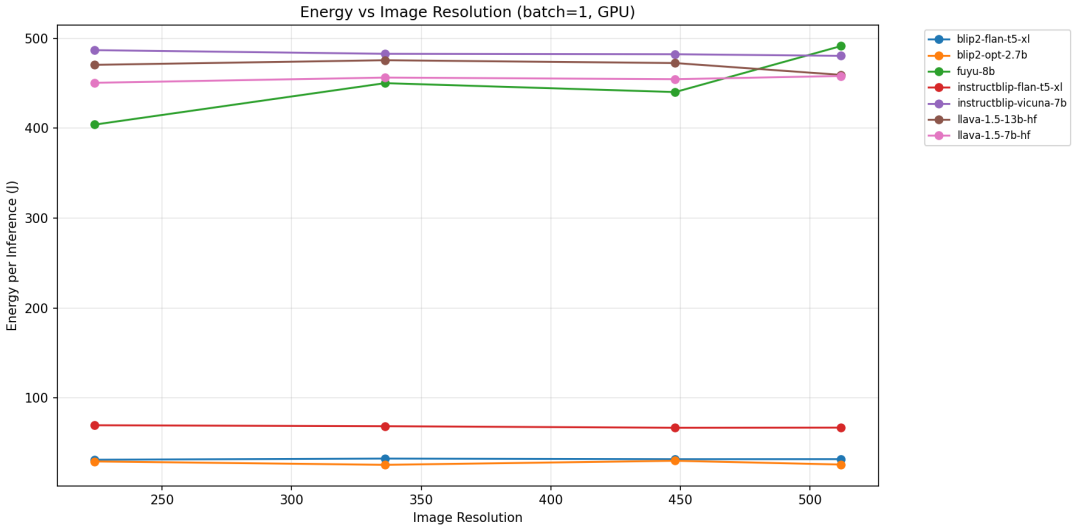

Figure 9: Energy vs. resolution.

Same story as latency vs. resolution: flat lines everywhere. The energy cost per inference does not change with resolution. Again, this confirms that the vision encoder is not the bottleneck.

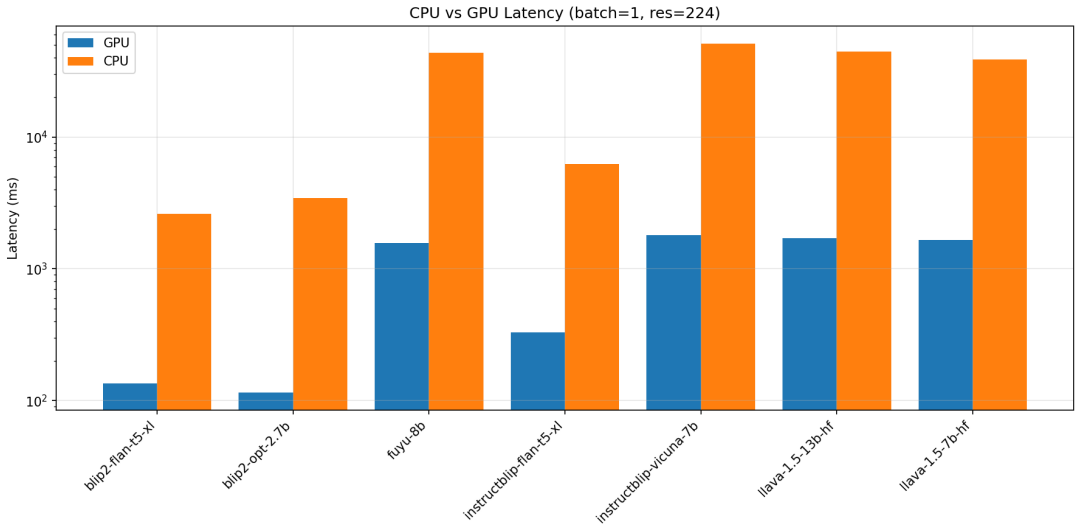

Figure 10: CPU vs. GPU latency (log scale).

GPU is 15–30× faster than CPU for all models. The smallest gap is on `blip2-flan-t5-` (136 ms GPU vs. about 2.3 s CPU, roughly 17×). The biggest gap is on the 7B+ models where CPU latency reaches 30–50 seconds, making CPU inference completely impractical for these sizes.

5 Experiment 2: per-component breakdown

The main project question is *which parts of the model need the most optimization in terms of time and energy?* The baseline charts above show that the language decoder correlates with latency, but that is parameter-count evidence, not runtime evidence. In this section we measure two complementary things directly.

5.1 Method

Top-level submodules (`vision_model` / `vision_tower` / `vision_embed_tokens`, `qformer` / `multi_modal_projector`, and `language_model`) are instrumented with PyTorch forward hooks and CUDA events. For each model we run `generate(max_new_to` three times at the baseline config and sum CUDA-event time inside each submodule. Separately, we estimate prefill and per-token decode latency by running `generate` with `max_new_tokens=1` and `max_new_tokens=50` and solving $t(N) \approx t_{\text{prefill}} + N \cdot t_{\text{decode}}$.

5.2 Component share of inference time

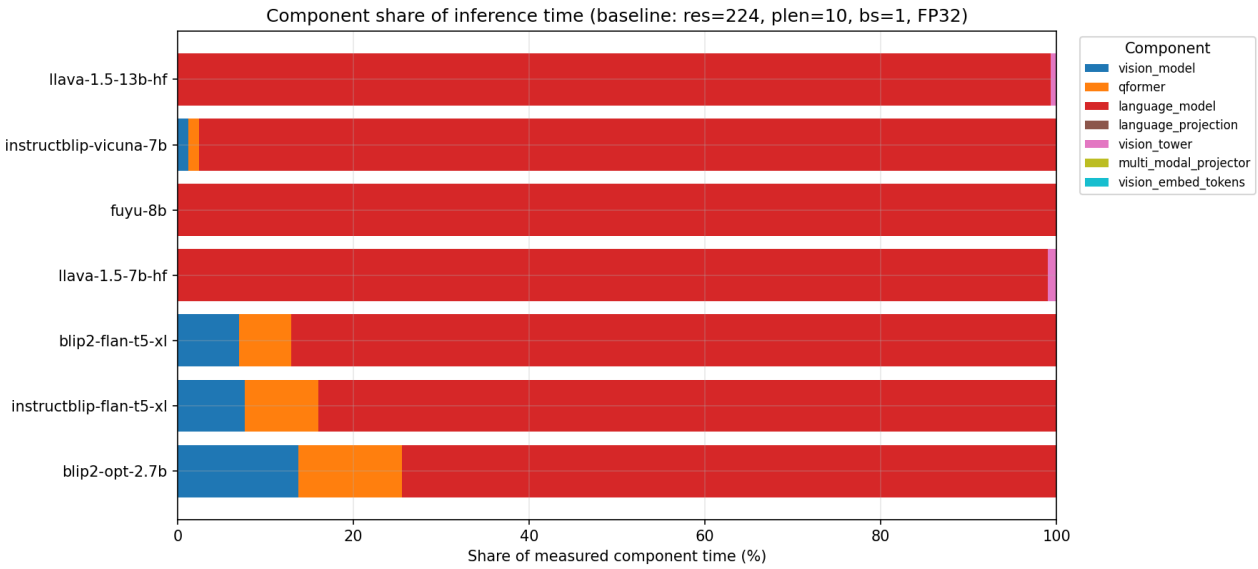

Figure 11: Share of measured forward-pass time spent inside each top-level submodule, at baseline (res=224, plen=10, bs=1, FP32, GPU).

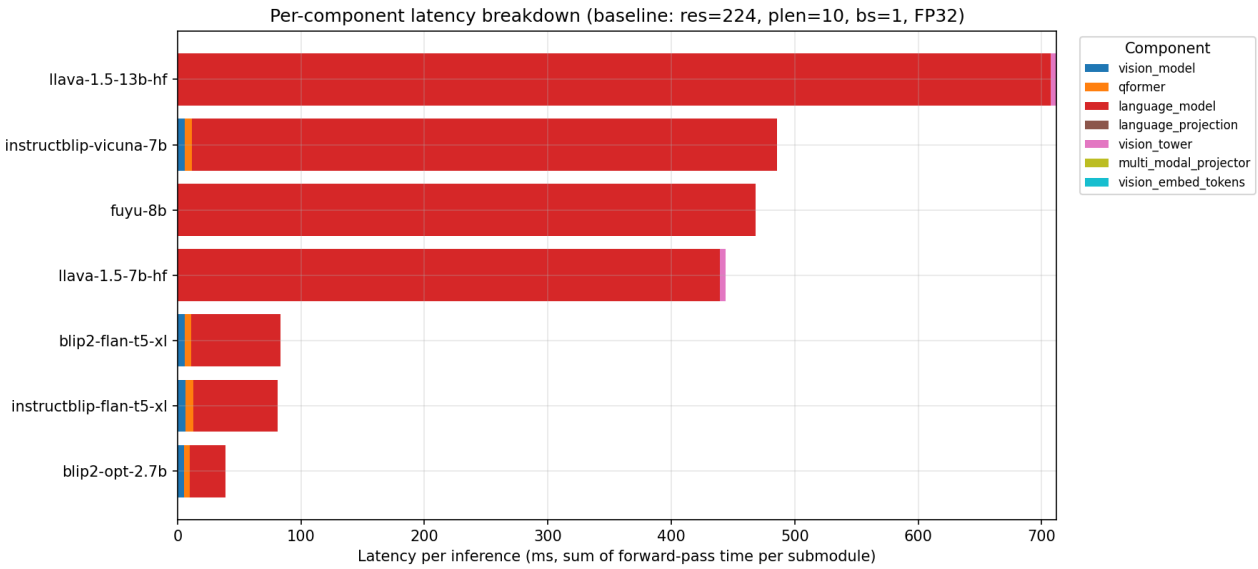

Figure 12: Same data as Figure 11 but in absolute milliseconds.

The language decoder dominates inference time, but the share depends sharply on model size. Measured decoder share: `blip2-opt-2.7b` 74 %, `blip2-flan-t5-xl` 87 %, `instructblip-flan-t5-xl` 84 %, and the 7B+ models (`instructblip-vicuna-7b`, `llava-1.5-7b`, `llava-1.5-13b`, `fuyu-8b`) all sit at 98–99.99 %. For the small BLIP-family the vision encoder + Q-former still accounts for roughly 15–25 % of forward time, so ViT-level optimizations are not pointless there. For everything $\geq$7B, the decoder is essentially the only thing that matters. This also matches the resolution sweep from Experiment 1: latency is flat in resolution because the vision encoder is a small fraction of the total, and the fraction only shrinks as models get larger.

5.3 Prefill vs. per-token decode

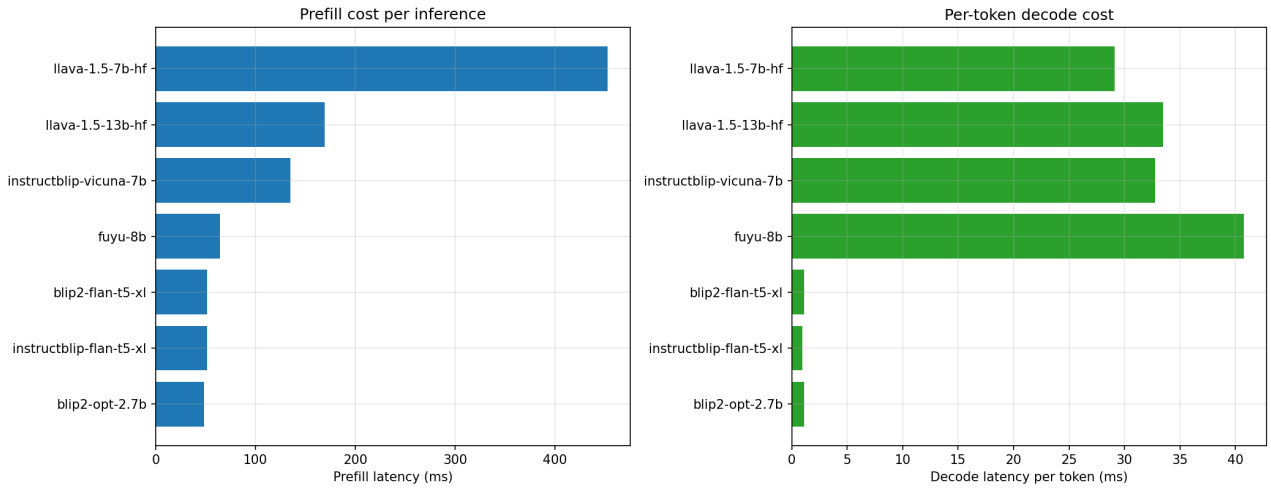

Figure 13: Prefill cost vs. per-token decode cost for each model. Baseline (FP32) except `llava-1.5-13b` which is loaded in FP16 because 48 GiB is insufficient for FP32 weights.

Dtype asymmetry and FP16 sanity check. In the chart above, `llava-1.5-13b` appears faster than `llava-1.5-7b`, which is suspicious. The reason is that 13B is loaded in FP16 (it does not fit in FP32 on a 48 GiB card) while 7B runs at baseline FP32. Running 7B in FP16 for a direct comparison confirms the dtype is what explains the gap: prefill drops from 432 to 94.5 ms (4.6×) and per-token decode from 29.5 to 15.3 ms (1.9×). Prefill wins more than decode because prefill attention is memory-bandwidth bound on Ada, and FP16 halves weight/activation traffic; decode is mostly matmul against a KV cache, closer to the theoretical 2× ceiling. In FP16 the expected size ordering is restored: 7B (94.5 ms prefill, 15.3 ms/token) is cleanly faster than 13B (169 ms prefill, 33 ms/token).

This split tells us *where inside the decoder* to focus:

- If prefill dominates total latency, attention over the prompt is the bottleneck. Attention-only optimizations like FlashAttention-2, paged KV cache, or chunked prefill will help the most.
- If per-token decode dominates at `max_new_tokens=50`, the autoregressive loop is the bottleneck. Speculative decoding, smaller draft models, or quantized KV cache are the right tools.

For the models measured here, prefill ranges in the tens to low hundreds of ms and decode in the 1–20 ms/token range. At `max_new_tokens=50` the decode portion equals roughly 50× the per-token cost, so whichever term is larger at that point in the chart is the dominant cost for this workload. For workloads that generate shorter answers (e.g. 5–10 tokens) prefill is more important; for longer generations (captioning, explanations), decode dominates quickly.

5.4 FlashAttention-2 contribution

The task description lists FlashAttention-2 as an optimization axis to evaluate. In this project FA2 is delivered via PyTorch's `scaled_dot_product_attention`: on Ada in FP16 the SDPA dispatch selects a Flash-Attention-2 kernel under the

hood. The standalone `flash-attn` pip package could not be built in this environment — it requires `nvcc` $\geq$ 11.7 and the system only has 11.5 — so we use the torch-native path, which Transformers 5.x already enables by default (`attn_implementation="sdpa"`).

To isolate FA2's contribution we re-ran two models in FP16 with `attn_implementatic` (the reference quadratic-memory attention) and compared against the SDPA default.

Table 7: Prefill and per-token decode with FA2-enabled SDPA vs. eager attention. Baseline config (res=224, plen=10, bs=1, FP16). Values are averaged over 7 timed runs after 3 warmup runs.

Model	SDPA prefill (ms)	eager prefill (ms)	**FA2 speedup**	SDPA decode/t
`llava-1.5-7b`	94.5	105.4	1.11×	15.30
`llava-1.5-13b`	169.0	181.3	1.07×	33.00

FA2 gives a 7–11 % prefill speedup at baseline and essentially no decode speedup (within measurement noise). The reason is sequence length: at plen=10 the LLaVA input is ≈ 586 tokens (10 text + 576 image patch tokens). FA2's benefit over eager attention comes from avoiding the $O(N^2)$ intermediate attention matrix materialization; at $N \approx 600$ that matrix still fits comfortably in cache, so the memory-access win is modest. The gain should scale with longer prompts, higher resolutions, or larger batch sizes — for workloads with N in the few-thousand-token range (long conversations, multi-image prompts, high-res captioning) the FA2 speedup on prefill would be substantially larger. Decode speed is unaffected because during single-token autoregressive steps the attention is over the KV-cache only, one query vector at a time, and the quadratic-memory concern does not apply.

Practical implication: for the short-answer VQA workloads profiled here, FA2 alone is not a meaningful lever. For long-context workloads it would be. The bigger win across the board remains the FP16 dtype flip itself (4.6× prefill for llava-7b) rather than the attention-kernel choice within FP16.

5.5 Takeaways

1. The LLM decoder is the optimization target for every 7B+ model (98–99.99 % of forward time). For models under 4B, vision + Q-former is still 15–25 % and is worth optimizing, but even there decoder is the majority.
2. For short answers (VQA-style, 5–10 tokens) optimize prefill: FlashAttention-2, chunked prefill.
3. For long answers (captions, explanations, ≥ 50 tokens) optimize decode: speculative decoding, INT8/INT4 weight quantization, KV-cache quantization.
4. FP16 alone gives 4.6× on prefill and 1.9× on decode for `llava-1.5-7b`. FlashAttention-2 on top of FP16 is only a 1.07–1.11× prefill speedup at baseline short prompts, because the sequence length is not large enough to stress eager attention; the gain would scale with longer contexts.
5. `fuyu-8b` has the highest per-token decode cost (41 ms) despite being 8B, because it has no separate vision tower: image patches are injected as

input tokens, which inflates the context length and therefore the KV cache at every decode step.

6 Known limitations

Table 8: Model-specific problems we ran into.

Model	Problem	Affected runs
adept/fuyu-8b	FP16 causes a Float/Half dtype mismatch	3
adept/fuyu-8b	Processor returns lists when batching	9
blip2-flan-t5-xl	torch.compile fails on T5 architecture	3
instructblip-flan-t5-xl	torch.compile fails on T5 architecture	3

Other things worth noting:

- Changing the input resolution has almost no effect on BLIP-2 and InstructBLIP latency. These models resize to a fixed internal resolution inside the vision encoder.
- FLOPs numbers are only directly measured (calflops) for some models. The T5-based models use a rough $2\times$ params estimate, so those numbers are not directly comparable.
- FP16 actually slows down the T5-based models because of dtype casting overhead inside the T5 forward pass.

7 Conclusions

Based on both experiments (baseline sweeps and per-component breakdown):

1. `blip2-flan-t5-xl` is the best all-rounder: lowest average WER (1.21), 136 ms latency, 31 J energy.
2. The smaller models (BLIP-2, InstructBLIP-T5) are about an order of magnitude faster and cheaper to run than the 7B+ models.
3. The 7B+ models (LLaVA, Fuyu, InstructBLIP-Vicuna) have WER in the 6–9 range at $>$1.5 s latency and $>$400 J per inference. The quality gain over the lightweight models does not justify the cost.
4. For deployment, `blip2-flan-t5-xl` or `blip2-opt-2.7b` are the obvious choices.
5. The LLM decoder dominates runtime for every model measured. Vision encoder and projector account for under 30 % of time even on the largest inputs. Any serious optimization effort should target the decoder (FlashAttention-2 for prefill, speculative decoding or quantization for decode).

References

[1] J. Li, D. Li, S. Savarese, S. Hoi, “BLIP-2: Bootstrapping Language-Image Pre-training with Frozen Image Encoders and Large Language Models,” *arXiv:2301.12597*, 2023.

[2] W. Dai, J. Li, D. Li, A. M. H. Tiong, J. Zhao, W. Wang, B. Li, P. Fung, S. Hoi, "InstructBLIP: Towards General-purpose Vision-Language Models with Instruction Tuning," *arXiv:2305.06500*, 2023.

[3] H. Liu, C. Li, Y. Li, Y. J. Lee, "Improved Baselines with Visual Instruction Tuning," *arXiv:2310.03744*, 2023.

[4] H. Laurençon, L. Tronchon, M. Cord, V. Sanh, "What matters when building vision-language models?" *arXiv:2405.02246*, 2024.

[5] Adept AI, "Fuyu-8B: A Multimodal Architecture for AI Agents," *adept.ai blog*, 2023.